

CareerCraft AI: Resume Role Classifier and Job Match Bot

Machine Learning Mini Project Report

Prepared by: Usmeet, Second-Year Computer Engineering, SPPU Guide / Teacher:
_____ | Roll No.: _____ | Date: October 2026

Concept inspiration: the CareerCraft AI idea from Manasvi Sawant's Gen AI Academy deck. The machine learning model, code and bot in this report were built separately for this project.

1. Problem Statement

When a person applies for a job, two questions matter. Does my resume fit this kind of role? And which skills does the job ask for that my resume does not show? Answering these by hand for every job posting is slow. This project builds a system that reads a resume, predicts the job category it belongs to, and compares it with a job description to list the skills the resume is missing.

2. Objectives

- Train a machine learning model that classifies a resume into one of 24 job categories.
- Evaluate the model honestly, including where it fails.
- Build a chat bot on top of the model that compares a resume with a job description and reports a match score, matched skills, and missing skills.
- Keep the code simple enough to explain line by line.

3. Dataset

The project uses the Kaggle Resume Dataset.

- About 2,484 resumes, each stored as plain text with a category label.
- 24 categories such as Information-Technology, HR, Chef, Engineering, Accountant and Teacher.
- The categories are **imbalanced**. Some have over 100 resumes while others, such as BPO, have only about 20. This matters because a model sees more examples of the large categories and tends to favour them.
- Only two columns are used: the resume text (`Resume_str`) and the label (`Category`). Rows with missing values are removed.

4. Methodology

The project follows a standard text classification pipeline.

Step	What happens	Why
1. Load data	Read the CSV, keep text and category, drop empty rows	Remove unusable data
2. Clean text	Lowercase, remove links, emails, numbers and symbols (keeping c++ and c#)	"Python" and "python" should be the same word
3. Split	80% training, 20% testing, stratified, fixed random seed	The model is tested on resumes it has never seen; stratify keeps category proportions equal
4. TF-IDF	Turn each resume into a list of 20,000 numbers	A model can only work with numbers
5. Train	Fit a Linear SVM on the training numbers	The model learns which words point to which category
6. Evaluate	Predict the hidden 20%, compute accuracy, report and confusion matrix	Measure how good the model really is
7. Save	Store the model and TF-IDF with joblib	The bot can reuse them without retraining

4.1 TF-IDF in plain words

TF-IDF stands for Term Frequency, Inverse Document Frequency. A word gets a high score if it appears often in one resume (term frequency) but rarely across all other resumes (inverse document frequency). So "payroll" scores high for an HR resume, while a common word like "the" scores near zero.

Settings used: English stop words removed, single words and pairs of words (unigrams and bigrams), the 20,000 most useful terms kept, and sublinear term frequency so that repeating a word many times does not dominate.

Important detail: the vocabulary is learned from the **training data only** (`fit_transform`) and then applied to the test data (`transform`). Learning it from all data would leak information from the test set into training.

4.2 Linear SVM in plain words

A Support Vector Machine finds the boundary that separates classes with the widest possible margin. In the linear version used here, each of the 24 categories gets a weight for every word. For a new resume, the model adds up the weights of the words it contains for each category, and the category with the highest total is the prediction. For example, "payroll" has a high weight for HR and "kitchen" has a high weight for Chef.

5. Model Selection and Results

Three models were compared on the same TF-IDF features: Logistic Regression, Linear SVM and Naive Bayes. Linear SVM performed best:

- **Accuracy: about 0.72** on the hidden 20% test set
- **Weighted F1-score: about 0.71**

Linear SVM suits this task because TF-IDF produces very high-dimensional, sparse data, which linear models handle well and quickly.

5.1 Reading the results

- Random guessing across 24 categories would give about 4% accuracy, so 72% shows the model learned real patterns.
- The confusion matrix shows which categories get mixed up. Categories with similar vocabulary, such as Engineering and Information-Technology, are the usual confusions.
- Small categories have lower recall because the model sees fewer examples of them. Adding `class_weight="balanced"` to the SVM, which gives small categories more weight during training, is a simple improvement and is used in the final training script.

6. The Chat Bot

The notebook model is wrapped in a Streamlit chat application.

How a user interacts:

1. Paste a resume, or upload a PDF or TXT file.
2. The bot shows the top 3 predicted roles with a rough confidence.
3. Paste a job description.
4. The bot shows the role of both texts, whether they match, a match score out of 100, the skills the resume already shows, the skills to add, and other missing keywords.

Commands: `new` starts over, `jd` swaps the job description, `help` shows instructions.

6.1 How the comparison works

- **Role prediction:** the same cleaning and TF-IDF steps as training, then the Linear SVM. A rough confidence comes from a softmax over the SVM scores. Below 30% the bot warns that the prediction is a guess.
- **Skill matching:** a hand-written list of about 150 skills (Python, SQL, AWS, Docker, cybersecurity and so on). The bot finds which skills appear in the job description and checks whether each appears in the resume.
- **Stemming:** a small suffix-stripping function treats "database" and "databases" as the same word, so the resume is not penalised for plural or tense differences.
- **Filler removal:** job-posting words such as "responsibilities", "hybrid" and "ideal candidate" are ignored because they are not skills.
- **Score:** 70% of the score is the share of the job's skills found in the resume, and 30% is a bonus when both texts map to the same role with enough confidence. This formula is a heuristic chosen for simplicity, not a validated measure.

6.2 Project files

File	Purpose
<code>train.py</code>	Trains the model from Resume.csv and saves it
<code>bot_core.py</code>	Cleaning, prediction, skill matching and score logic
<code>skills.py</code>	The skill list (easy to extend)
<code>app.py</code>	Streamlit chat interface
<code>chat_cli.py</code>	Terminal version of the same bot

7. Sample Output

Test 1. A second-year engineering student's resume (Python, SQL, machine learning, Java projects) against a Data Engineering Intern posting.

- Score: 68/100
- Skills already shown: communication, data engineering, git, google cloud, machine learning, python, sql
- Skills to add: airflow, bigquery, data visualization, docker, etl, spark

Test 2. The same resume against an IT / system administration posting.

- Score: 30/100, role match: no

- Skills already shown: cloud computing, cybersecurity, sql
- Skills to add: aws, networking, system administration, technical support

Test 3. The same resume against a Cloud Security Engineer posting.

- Score: 8/100, role match: no
- Only Linux matches; the posting asks for AWS, Azure, Kubernetes, Terraform, IAM and penetration testing

The scores fall in the order a person would expect: the closer the job is to the resume, the higher the score.

8. Limitations

- **Dataset coverage:** the dataset has no Data Science, AI/ML or student category, so a student's AI-focused resume is forced into the nearest category (Engineering). The bot flags low confidence in such cases.
- **Word counting, not meaning:** TF-IDF does not understand context. "Used Docker daily" and "never used Docker" look the same, and abbreviations such as "K8s" will not match "Kubernetes".
- **Keyword stuffing:** a resume can raise its score just by listing keywords.
- **Hand-written skill list:** the bot only finds skills that are in `skills.py`.
- **Unvalidated score:** the 0-100 score is a formula, not checked against real hiring decisions.
- **Reading limits:** PDFs with multiple columns or scanned images may not extract correctly.
- **Not a hiring tool:** the bot cannot judge impact, seniority or writing quality, and models trained on past resumes can carry bias. It should help candidates improve a resume, never reject candidates.

9. Real-World Use and Future Work

Where it is useful today: a candidate checking a job posting against their resume before applying, and as a teaching example of a full machine learning workflow from data to a working application.

Possible improvements:

1. Replace TF-IDF with sentence embeddings so that matching depends on meaning, not just shared words.
2. Add a large language model step that suggests how to rewrite resume bullets.

3. Use a standard skills taxonomy (such as O*NET or ESCO) instead of a hand-written list.
4. Collect 30 to 50 real resume and job pairs to check that the score matches human judgement.
5. Deploy online (Streamlit Community Cloud or Hugging Face Spaces) so anyone can try it.

10. Tools and Libraries

Python, pandas (data handling), scikit-learn (TF-IDF, Linear SVM, metrics), matplotlib (confusion matrix), joblib (saving the model), Streamlit (chat interface), pypdf (reading PDF resumes), Google Colab (training environment).

11. Likely Viva Questions and Answers

Why did you clean the text? So the same word in different cases or with symbols is treated as one word, which reduces noise.

Why split into training and testing data? To measure performance on resumes the model has never seen. Testing on training data would give a falsely high score.

What does stratify do? It keeps the same category proportions in the training and test sets, which matters when categories are imbalanced.

Why TF-IDF and not simple word counts? Word counts over-reward common words. TF-IDF boosts words that are distinctive to a resume and reduces words that appear everywhere.

Why Linear SVM? It was the best of the three models tried, and linear models work well on sparse, high-dimensional text data.

Why fit TF-IDF only on training data? Fitting on all data would let test information leak into training and make the evaluation too optimistic.

Is 72% accuracy good? With 24 categories, chance is about 4%, so it is strong. It is not perfect because some categories share vocabulary and some have few examples.

What is class imbalance and what did you do about it? Some categories have far fewer resumes than others. I added balanced class weights so that small categories count more during training.

What is the biggest weakness? The dataset has no AI/ML category and the method counts words without understanding context. Embeddings would address both partly.

How is the match score calculated? 70% skill coverage plus 30% for a confident role match. It is a heuristic, and I say so openly.

12. Conclusion

This project shows the full path from raw data to a usable tool: cleaning text, turning it into numbers, training and comparing models, evaluating honestly, and wrapping the result in a chat application that gives a candidate practical, actionable feedback. The model reaches about 72% accuracy across 24 categories, and the report identifies exactly where it is weak and how it could be improved.